

1 Euler Tour

```

1  int timer = 0;
2  // Queries on subtree range [in,out)
3  function<void(int,int)> euler_tour = [&](int u, int
    p) -> void {
4      in[u] = timer++;
5      for(int v : adj[u])
6          if( v != p )
7              euler_tour(v,u);
8      out[u] = timer;
9  };
10 euler_tour(1,0);
11 //===== Queries with Fenwick Tree
12 BIT<ll> bit(V);
13 for(int u = 1; u <= V; u++) bit.update(in[u]+1,v[u])
    ;
14 // Update 'node' to 'val' (set operation)
15 bit.update(in[node]+1,val-v[node]);
16 v[node] = val;
17 // Query (ET is 0-idx, but BIT 1-idx)
18 cout << bit.query(in[node]+1,out[node]) << endl;
19 //===== Queries with Segment Tree
20 vll flat(V+1);
21 for(int i = 1; i <= V; i++)
22     flat[ in[i] ] = v[i];
23 Segment_tree st(V);
24 st.build(flat,1,0,V-1);

```

```

25 st.update(1,in[node],0,V-1,val);
26 st.query(1,in[node],out[node]-1,0,V-1);

```

Listing 1: Euler Tour implementation, runs in $O(|V| + |E|)$. Works with an 1-indexed logic, but the time mapping is 0-indexed. Brings necessary logic to work with BIT and ST templates.

2 Coordinate Compression

```

1 //Previously read vector<int> a(n)
2 vector<int> coords = a;
3 sort(all(coords));
4 coords.erase(unique(all(coords)), coords.end()); //
    erases duplicates
5 //Reassings every value in vector "a" to the new
    corresponding value
6 for(int &x : a) x = lower_bound(all(coords), x) -
    coords.begin();

```

Listing 2: Coordinate Compression implementation for a vector of integers. Replaces elements in the original vector to newly assigned minimal numbers. Runs in $O(n \log(n))$.

3 Sqrt Decomposition

3.1 Mo's Algorithm

```

1 struct Query{
2     int l, r, idx;

```

```
3 };
4 //vector queries ya leido y 0-indexado anteriormente
5 int block_size = (int)sqrt(n);
6 auto mo_cmp = [&](Query a, Query b) { //custom
    cmptor
7     int block_a = a.l / block_size;
8     int block_b = b.l / block_size;
9     if(block_a != block_b) return block_a < block_b;
10    if(block_a & 1) return a.r < b.r; //zig-zag
    sorting
11    return a.r > b.r;
12 };
13 sort(all(queries), mo_cmp);
```

Listing 3: Implementation of Mo's Algorithm for offline query sorting. Efficient in static range queries problems, since it's easier to implement than a segment tree. Runs in $O((n + q)\sqrt{n} \cdot F)$, where F stands for the time complexity of an update (usually $O(1)$ or $O(\log(n))$)